DBT PORTFOLIO PROJECT

LAYER: STAGING

DBT Staging Model

Staging | Materialized View | Type Casting | Data Quality | Snowflake

Project Summary

The type-safe staging layer for the raw healthcare claims source. This model applies explicit Snowflake type casts to every column (::date, ::float, ::int) ensuring clean, consistent data types flow into all downstream intermediate and mart models. It is the single clean entry point for the entire pipeline.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	stg_healthcare_claims
Layer:	Staging
Materialization:	view (lightweight -- rebuilt on every run)
Source:	{{ source('healthcare', 'claims') }}
Key Output:	14 type-cast columns ready for all downstream models
Audience:	Downstream DBT models (not exposed directly to BI tools)

2 BUSINESS PROBLEM

Raw source tables in Snowflake often store all columns as VARCHAR or VARIANT to accommodate flexible ingestion. If downstream models don't cast types explicitly, date arithmetic fails, averages on strings return errors, and fraud flags behave as text instead of integers. This staging model eliminates all of those failure modes in one place.

- period_date::date -- ensures all window functions and date joins work correctly.
- metric_value::float and return_value::float -- enables AVG(), STDDEV(), and arithmetic.
- claim_volume::int and days_to_payment::int -- prevents float drift in integer aggregations.
- is_fraud_flag::int -- ensures binary flag behaves correctly in SUM() and COUNT() expressions.
- y_value, x1_value, x2_value::float -- regression models require numeric precision.

3 TECHNICAL EXPLANATION

- Materialized as 'view' -- no storage cost; always reflects the latest source data on query.
- Uses {{ source('healthcare','claims') }} -- ties into dbt lineage and source freshness.
- Snowflake :: cast operator used throughout -- concise and idiomatic for Snowflake SQL.
- No filtering or transformation logic -- staging models are intentionally simple and transparent.
- All 14 source columns are carried through -- downstream models SELECT only what they need.

4 FULL MODEL CODE

```
{{ config(materialized='view') }}

select
    entity_id,
    payer,
    state,
    specialty,
    period_date::date         as period_date,
    metric_value::float       as metric_value,
    claim_volume::int         as claim_volume,
    return_value::float       as return_value,
    denial_rate::float        as denial_rate,
    days_to_payment::int      as days_to_payment,
    is_fraud_flag::int        as is_fraud_flag,
    y_value::float            as y_value,
    x1_value::float           as x1_value,
    x2_value::float           as x2_value

from {{ source('healthcare', 'claims') }}
```

5 EXAMPLE OUTPUT

entity_id	payer	state	period_date	metric_value	return_value	is_fraud_flag
E-001	BlueCross	AR	2025-01-01	4250.75	0.032	0
E-002	Aetna	TX	2025-01-01	3180.20	0.018	0
E-003	UnitedHealth	MO	2025-01-01	5920.10	0.041	1